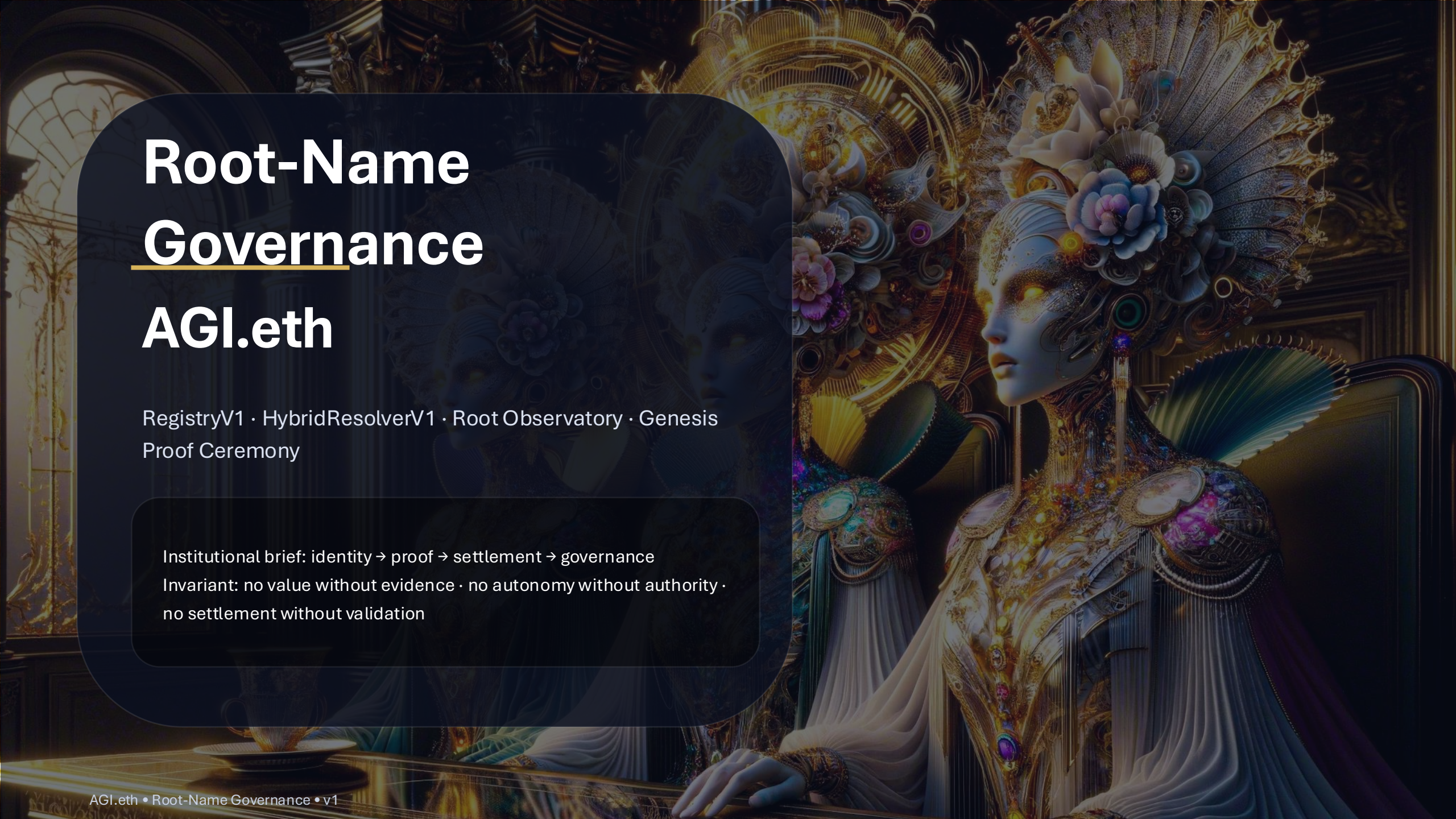


# Root-Name Governance

# AGI.eth

RegistryV1 · HybridResolverV1 · Root Observatory · Genesis Proof Ceremony

Institutional brief: identity → proof → settlement → governance
Invariant: no value without evidence · no autonomy without authority · no settlement without validation

# Executive Charter



## Identity

- Roles bound to keys (ENS/DID).
- Accountability by default.
- Segmented by role roots (agent / node / club).

## Evidence

- Deterministic runs emit signed proof bundles.
- Replay beats rhetoric (audit as replay).
- Proofs are content-addressed.

## Settlement

- Escrow releases only after validation.
- Receipts are settlement-grade artifacts.
- Disputes resolved by replay evidence.

## Governance

- Policy gates, upgrades, emergency brakes.
- Operator override is final (autonomy is bounded).
- Change control is timelocked and observable.

Invariant: no value without evidence · no autonomy without authority · no settlement without validation

# Namespace Grammar

## <entity>.(<env>.)<role>.agi.eth

role ∈ {club, agent, node} · env ∈ ENV_SET (optional; e.g., alpha, …)

### Role patterns (global + env-scoped)

| | | |
|---|---|---|
| Validators: | *.club.agi.eth | *.alpha.club.agi.eth |
| Agents: | *.agent.agi.eth | *.alpha.agent.agi.eth |
| Nodes: | *.node.agi.eth | *.alpha.node.agi.eth |
| Sovereigns: | *.agi.eth | *.alpha.agi.eth |

### Examples (env = alpha)

Validator: alice.club.agi.eth
alice.alpha.club.agi.eth

Agent: helper.agent.agi.eth
helper.alpha.agent.agi.eth

Node: gpu01.node.agi.eth
gpu01.alpha.node.agi.eth

Sovereign: deployer.agi.eth
deployer.alpha.agi.eth

# Recognition & Scope

| Name form | Class | Recognized in any env? | Developed / governed where? |
|---|---|---|---|
| entity.role.agi.eth | Global role identity | Yes | Global role namespace + each env policy |
| entity.env.role.agi.eth | Env-scoped identity | No | Only inside env.agi.eth |
| env.role.agi.eth | Env-role mountpoint | No | Only inside env.agi.eth |
| role.env.agi.eth | Optional alias mountpoint | No (default) | Only inside env.agi.eth (optional recognition) |

## Best-practice guardrails

- Reserve {agent, node, club} under every env.agi.eth (prevents confusion with businesses).
- Optional aliases are conveniences; “official” status is granted only via env registry (recognizedAliases).
- Keep semantics stable; push churn into resolvers (wildcards + CCIP-Read), not naming.

# Root Custody & Change Control



## RootOwner (cold custody)

- Highly-distributed multisig.
- Controls: owner, resolver, subname policy.
- Never used for routine operations.

## Timelocked structural changes

- Resolver swaps, env creation, role-root reassignment.
- Public delay + on-chain intent trail.
- Prevents rushed governance attacks.

## Operational multisig

- Limited scope: signer rotations, gateway URLs.
- Cannot transfer ownership.
- Fast response without systemic risk.

## Emergency guardian

- Can trigger “fail-closed” mode.
- Freezes mutable endpoints; status=emergency.
- Designed to stop blast radius, not seize control.

Renewals are existential: automate monitoring · fund renewals · publish runbooks

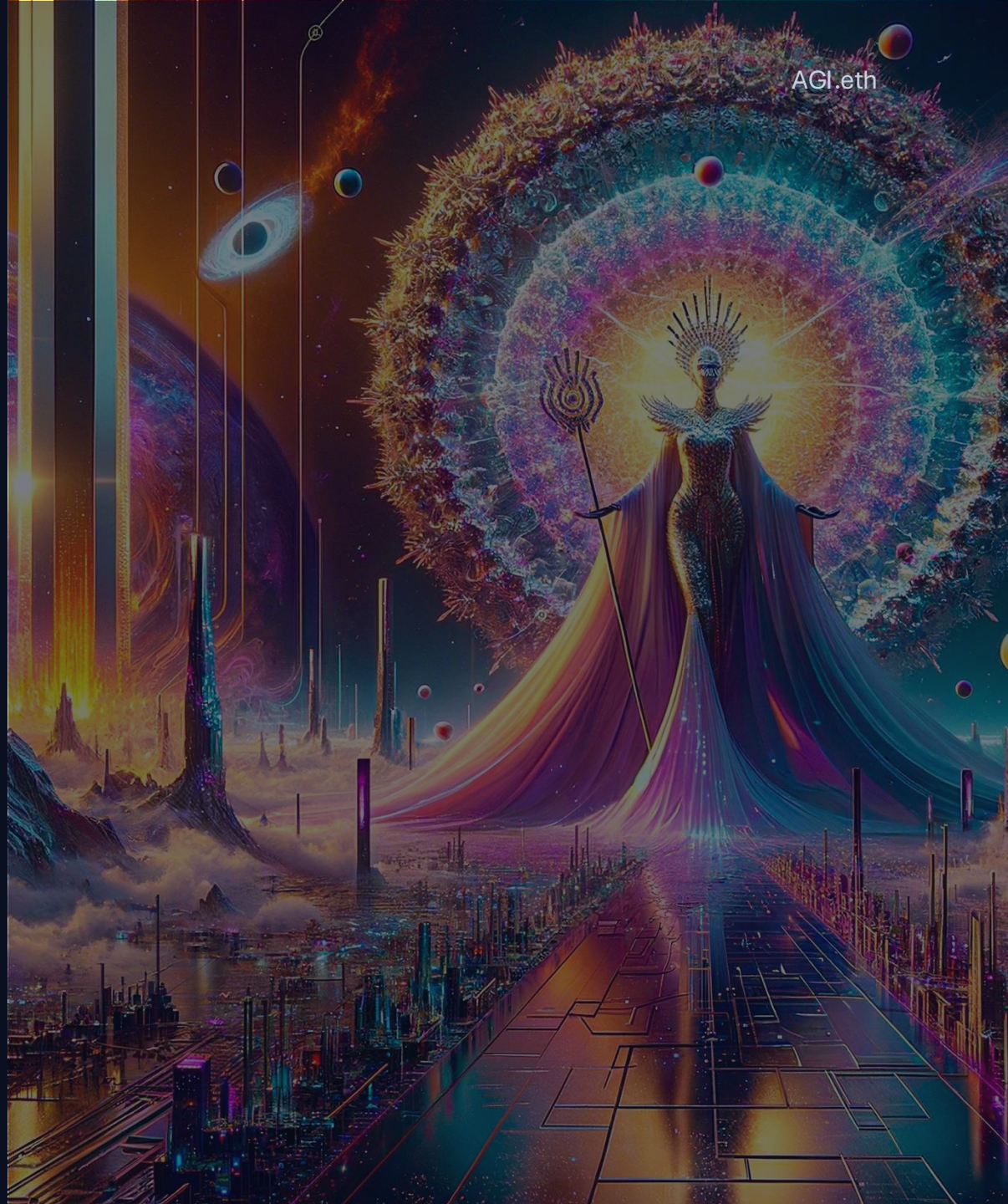
# RegistryV1 — Machine-Checkable Genome

Purpose: publish ENV_SET + canonical packages + recognizedAliases as the source of truth for official recognition.

```
{
  env, state,
  canonicalPackage: {
    root, agentMount,
    nodeMount, clubMount
  },
  recognizedAliases: [ ... ]
}
```

## V1 requirements

- Alpha first: ENV_SET = { alpha }
- rootLastChangeHash updates on any mutation
- Alias recognition is explicit (no self-assertion)
- Resolvers must fail-closed for unrecognized env/aliases




Genome → enables deterministic recognition at scale

# HybridResolverV1 — Wildcard + Signed Resolution



## Principles

- Honor on-chain subnames when they exist (sovereign overrides).
- Wildcard fallback for everything else (no per-agent writes).
- CCIP-Read gateways return signed answers; resolver verifies.
- Segmented blast radius: deploy separate resolvers for agent.*, node.*, club.*
- Registry-gated semantics: unrecognized env/alias ⇒ fail-closed.

## Signed response binding

Signature must bind: chainId · resolver address · request calldata · result · expiry.

## Resolution flow (concept)

| | |
|---|---|
| 1 | ENS Registry<br>(owner + resolver) |
| 2 | Role root resolver<br>(agent / node / club) |
| 3 | On-chain overrides<br>(if present) |
| 4 | CCIP-Read gateway<br>(OffchainLookup) |
| 5 | Signed response<br>(allowlisted signer) |
| 6 | Client receives<br>addr/text/contenthash |

# Metadata Surface: Text Records + Signed Discovery



## Standardized text keys (v1)

```
agi:type        = agent | node | validator | sovereign
agi:env         = alpha | …
agi:id          = stable unique identifier
agi:controller  = controlling address / DID
agi:status      = active | draining | disabled |
slashed
agi:endpoints   = endpoints JSON or URL
agi:capabilities= CID/URL
agi:provenance  = proof bundle pointers
agi:discovery   = signed discovery doc (JWS or
detached sig)
```

Design: stable semantic identity · dynamic endpoints behind verified resolution

## Two-layer signature model

1) Resolver-level (CCIP-Read)
— namespace operator attests the record

2) Entity-level (discovery doc)
— agent/node attests its own endpoints + keys

Both are replayable; both are verifiable.

### Example: discovery pointer

```
{
  "kid": "node-key-2026-01",
  "endpoints": ["https://…", "wss://…"],
  "capabilities": "ipfs://…",
  "sig": "0x…"
}
```

# Root Observatory — status.agi.eth



## World-verifiable dashboard

Shows (minimum):

- ENS owner + resolver for agi.eth
- Role resolvers for agent.*, node.*, club.*
- Gateway URLs + signerSetHash + emergency flag
- RegistryV1 ENV_SET + env states
- rootLastChangeHash + last update block

Outcome: anyone can verify the organism's current state without trusting an operator.

## Machine-verifiable status.json

```
{
  "schema": "agi.rootstatus.v1",
  "chainId": 1,
  "root": "agi.eth",
  "ens": { "owner": "0x…", "resolver": "0x…" },
  "registry": { "address": "0x…", "rootHash": "0x…" },
  "roleResolvers": { "agent": "0x…", "node": "0x…",
"club": "0x…" },
  "envSet": [ { "env": "alpha", "state": "ACTIVE" } ],
  "snapshotHash": "0x…",
  "signature": "0x…"
}
```

# Genesis Proof Ceremony

Run as one AGI Job → produce a signed Root Snapshot Pack

## Phase A — Build + Proposal Pack

- Compile & deploy RegistryV1 + resolvers
- Generate Safe/Timelock transaction bundle
- Produce TargetStateHash (intended end state)
- Publish build SBOM + deterministic seeds

## Phase B — Execute + Verify + Snapshot

- RootOwner executes the bundle
- Worker verifies on-chain state matches TargetStateHash
- Generate RootSnapshotPackV1 (CID + snapshotHash)
- Validators run commit-reveal → quorum certificate
- Escrow settles only after validated proof

If it cannot be replayed, it does not settle.

# Root Snapshot Pack V1



## Pack layout (content-addressed)

```
RootSnapshotPackV1/
  manifest.json + signatures
  chain/   txs.json · receipts · events
  ens/     owner · resolvers · pointers
  genome/  RegistryV1 state + rootHash
  metabolism/ resolver configs + signerSetHash
  constitution/ contenthash targets
  observatory/ status.json + signature
  proofs/  SBOM · container digest · logs · replay
  attestations/ commit-reveal + quorum certificate
```

## Acceptance tests (minimum)

- registry.agi.eth → RegistryV1
- role resolvers set (agent/node/club)
- alpha env state matches policy
- reserved labels under alpha.agi.eth
- constitution + status contenthash set
- snapshotHash reproducible
- worker/node signatures verify
- validator quorum certificate present

# Free-Energy Governance



### Minimize coordination energy

- Fewer on-chain writes
- Deterministic operations
- Clear change control
- Automation over meetings

### Minimize namespace entropy

- One canonical grammar
- Registry-based recognition
- Reserved labels
- Low ambiguity surfaces

### Push churn into resolvers

- Wildcards + CCIP-Read
- Signed discovery
- Fast endpoint rotation
- Stable trust anchors

## Dominant strategies + thermostats

- Workers earn only for validated proof.
- Validators are stake-bonded; misbehavior becomes negative-sum (slashing).
- Commit–reveal reduces herding/bribery leverage; disputes resolved by replay evidence.

Thermostats adjust parameters using observed signals (throughput, disputes, SLO drift):
Signals: validated work / epoch · dispute rate · SLO drift
Actuators: fee splits · burn fraction · tier multipliers · quorum/slashing

# Ship V1 as One Coherent Release



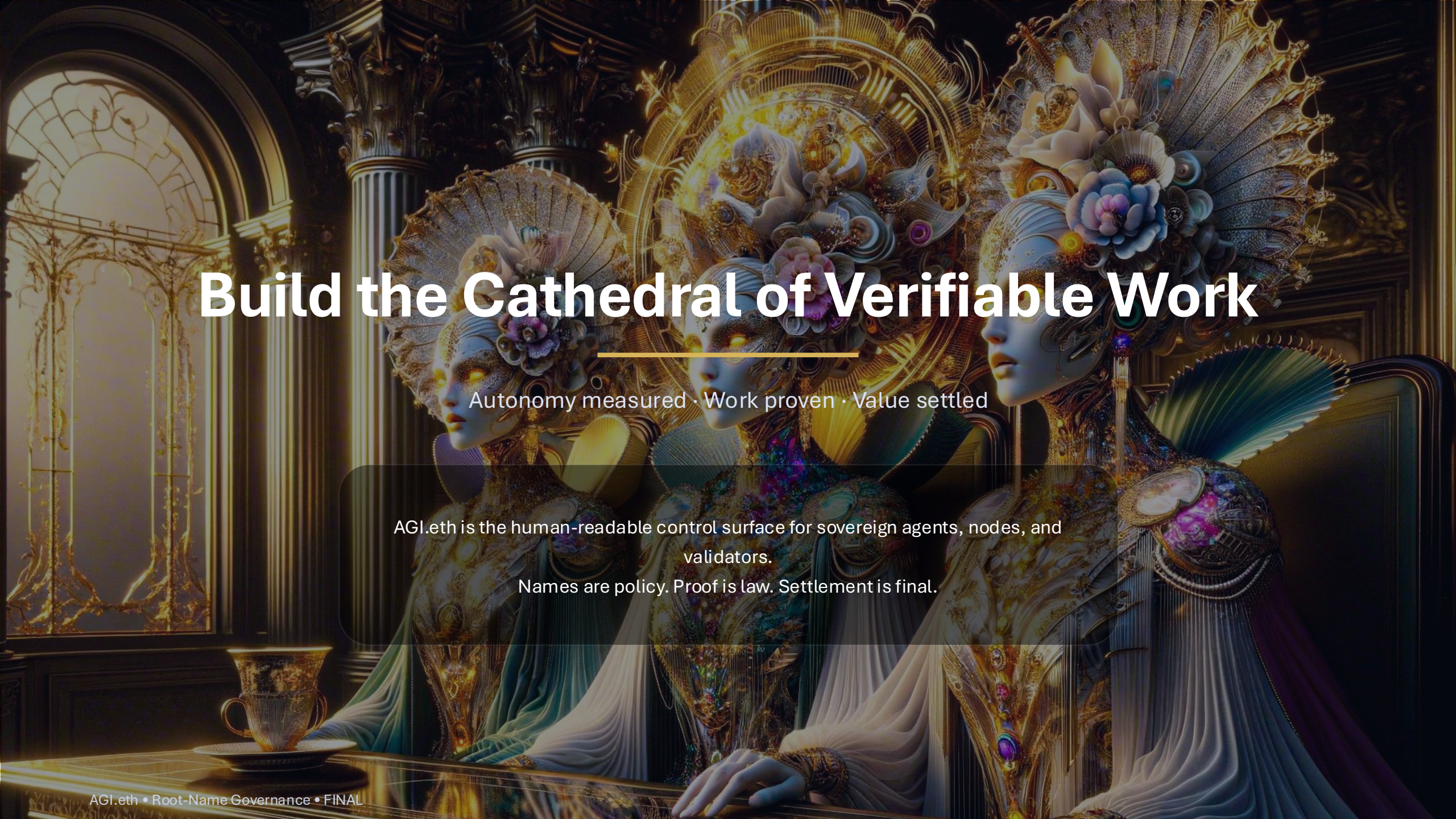
## Release checklist (single ceremony)

1. Deploy RegistryV1 (ENV_SET={alpha} + canonical packages + aliases)
2. Deploy HybridResolverV1 (3 instances: agent.*, node.*, club.*)
3. Point registry.agi.eth → RegistryV1 (addr + schema text record)
4. Set role roots (agent/node/club) resolvers → HybridResolverV1 instances
5. Publish constitution + status observatory (contenthash)
6. Reserve {agent,node,club} under alpha.agi.eth
7. Run Genesis Proof Ceremony AGI Job → RootSnapshotPackV1 + quorum cert

## "Shipped" when

- RegistryV1 is live + queried by clients
- Wildcard resolvers return verified records
- status.agi.eth shows owner/resolvers/signers/env set
- Genesis Snapshot Pack is published + signed
- Quorum certificate exists (commit-reveal)

Result: a namespace that behaves like an organism—genome, metabolism, immune system.



# Build the Cathedral of Verifiable Work

Autonomy measured · Work proven · Value settled

AGI.eth is the human-readable control surface for sovereign agents, nodes, and validators.
Names are policy. Proof is law. Settlement is final.